SHIELD-AI Reference Architecture Walkthrough

A presenter’s guide for technical audiences

Security Architecture Practice

September 1, 2026

Abstract
A detailed, presenter-ready walkthrough of the SHIELD-AI reference architecture for technical SMEs, refined through a Six Thinking Hats session. One coherent story: follow a single agent request through the five zones, four decision points, and the audit trail, anchored to real 2026 incidents. Includes the seven-act narrative, per-slide guidance, the fail-closed beat, the topology close, and handout guidance.

SHIELD-AI Reference Architecture Walkthrough

The one-sentence version

Every agent action passes four gates; every verdict is audited; if any gate or audit path fails, the action is blocked.

That sentence is the whole talk. The reference architecture is the sentence, unpacked into components. If the audience can repeat it after the session, the interaction story has landed.

This walkthrough was refined through a Six Thinking Hats session (White → Black → Yellow → Green → Red → Blue) run against the prior presentation’s failure: the individual elements were understood, the connective tissue was not. The hats identified the root causes (component-tour structure, diagram overload, no single through-line, control-ID firehose, two-audience collision, passive description, buried fail-closed payoff) and produced the narrative below.

How to use this document

This is a **presenter's guide**, not a slide deck. It gives you the story to tell, the order to tell it in, and the words to anchor each beat. The companion deck (`reference-architecture-walkthrough.pptx`) provides the slides; the source document is `framework/shield-ai-implementation-architecture.md` for any detail this guide does not carry.

- **Audience:** technical SMEs (security architects, platform engineers, AI engineers).
- **Length:** 30-35 minutes including the topology close.
- **Slides:** exactly two diagrams on slides (request flow, network boundary); the remaining five diagrams travel as handouts.
- **Golden rule:** narrate it from memory. If you can tell agent-7's story without the deck, the deck is support, not a crutch.

Act 0: The One-Sentence Version (60 Seconds)

Open with the sentence, then prove it: *"Every agent action passes four gates; every verdict is audited; if any gate or audit path fails, the action is blocked."*

Then the promise: *"The next thirty minutes are this sentence, unpacked. You will see one request travel through the whole system, and at each stop, a real 2026 incident explains why that component exists."*

Say it without slides. If the opening is narratable, the room trusts the rest.

Act 1: The Protagonist (3 Minutes)

Introduce **agent-7** and its user. One user, one agent, one request. Keep it concrete:

- A user in a business application asks agent-7 to summarise a thread and draft a reply.
- The request enters through the **AI gateway** (Zone 1): authentication, web-application filtering, intent filtering, rate limiting.

- The **orchestrator** (LangGraph, CrewAI, Semantic Kernel, or equivalent) dispatches the task to agent-7's sandbox (Zone 2).

What the audience should take away: the boundary of the system. Everything the user touches is Zone 1; everything the agent does happens inside a sandbox in Zone 2. The gateway is the front door, and it filters before the agent ever sees the request.

Presenter note: do not enumerate the eight component groups here. The audience will meet each one at the moment it matters. This is the discipline that fixes the component-tour failure.

Act 2: The Four Gates (10 Minutes)

This is the heart of the walkthrough. Every agent action (a tool invocation, a data access, a model call, a delegation) passes through four decision points evaluated by the OPA policy engine (Zone 3). **Anchor each gate to a real 2026 incident the audience already knows.** The incidents become the reason each gate exists.

P1: Tool Authorisation

Is this agent authorised to call this tool, within its registered scope?

- What it checks: agent identity (IAM-01), scope-bound delegation (IAM-02), per-tool scoped credentials (IAM-08), tool access control (TP-02).
- **Incident anchor: OpenClaw CVE-2026-25253 (CVSS 8.8).** A crafted link hijacked a loopback agent gateway, switched off approvals, and escaped to the host. The registration and scoping that P1 enforces is exactly the control that attack bypassed.
- **The gloss:** agent-7 may call `summarise_thread` because it is registered, versioned, and scoped to that agent. It may not call `drop_database`: that tool is not in the registry, and P1 denies.

P2: Data Access

What data may the agent retrieve, at what classification, under what conditions?

- What it checks: data classification (DS-01), attribute-based access control at retrieval (DS-04), data minimisation (DS-04).

- **Incident anchor: AWS Kiro CVE-2026-10591.** Hidden 1px white text on an API documentation page rewrote the agent's MCP server registry and ran attacker code on the host. The agent reached for data and configuration it should never have been able to touch. P2's classification and ABAC gates are the answer.
- **The gloss:** agent-7 may retrieve the thread because the documents are classified and the agent's attributes permit access. A document outside the agent's classification boundary is denied at retrieval, before it ever enters context.

P3: Human Approval

Does this action require a human approval gate?

- What it checks: human oversight (IAM-03).
- **The rule:** high-risk actions (code execution, financial writes, security-relevant changes) require an approval gate. The gate is infrastructure-enforced, not a polite request.
- **Incident anchor: the ABC News Australian case (INC-32).** A personal assistant booked classes months ahead and cancelled another member's waitlist position because no gate existed between the agent and the action. P3 is the gate that incident was missing.

P4: Output Validation

Is the tool response schema-valid and sanitised before it enters agent context?

- What it checks: tool output validation (TP-04), output scanning for PII and prompt leakage (DS-06), behaviour validation at the proxy (MS-05).
- **Incident anchor: implicit tool exposure (CVE-2026-26030, CVE-2026-25592).** A single prompt achieved code execution because an internal `DownloadFileAsync` helper was auto-registered as a callable tool. Outputs are structurally validated and sanitised before they enter agent context, so a malicious or malformed response cannot steer the agent.
- **The gloss:** the reply draft comes back from the tool; the gateway validates it against a known-good schema and sanitises it before the agent (or the user) ever sees it.

The rhythm: four gates, four incidents, one message. The architecture is the answer sheet to the incident ledger. Let the audience feel the pattern.

Presenter note: when you mention a control ID, gloss it. Never say “TP-08” alone; say “TP-08, the infrastructure-isolation control that blocks cloud control planes”. Taxonomy becomes texture, not noise.

Act 3: The Sandbox and the Sidecar (5 Minutes)

Where does enforcement actually live? In Zone 2, inside the sandbox, in a sidecar that carries the policy engine, the tool gateway, and the telemetry exporter.

- The sandbox: read-only root filesystem, seccomp/AppArmor profiles blocking mount/namespace/process-control syscalls, NetworkPolicy egress allow-list, resource quotas.
- The sidecar carries the OPA engine, so policy evaluation happens next to the agent, not in a distant service.
- **Runtime attestation:** sandbox mode is attested before each code-execution step (EE-06). No attestation, no execution. The sidecar fails closed: no policy, no audit, no action.

Incident anchor: the CrewAI fail-open chain (CVE-2026-2275/2286/2287). Prompt injection turned into host RCE because the framework *fell back* to a weaker sandbox when the configured one was unavailable. The attestation and fail-closed behaviour in this zone exist because that failure mode is real and documented.

What the audience should take away: enforcement is local, per agent, and it does not degrade. The sandbox is not a checkbox; it is the difference between an incident and a note in the audit log.

Act 4: The Data and Memory Plane (4 Minutes)

Zone 4: the RAG security layer and agent memory stores.

- Documents are classified at ingestion (DS-01).
- Retrieval enforces ABAC at query time with field-level redaction before context injection (DS-04).
- Agent memory is per-agent scoped, encrypted, and sanitised between sessions (DS-03).

- Outputs are scanned for PII, training data, and system-prompt leakage (DS-06).

Incident anchor: Ruflo/RufRoot CVE-2026-59726 (CVSS 10.0). An MCP-bridge RCE with persistent memory poisoning: the attacker's patterns were written into the learning store and steered every future user's outputs. "Patching doesn't undo poisoning." The memory-store integrity control (MS-10) exists because of this incident.

What the audience should take away: data is not a passive resource in an agent system; it is an attack surface. The plane between the model and the organisation's data is where classification, redaction, and memory hygiene live.

Act 5: The Audit Trail (4 Minutes)

Zone 5: telemetry and incident response. Every event from the journey so far (tool calls, data accesses, policy verdicts, model calls) was emitted as a structured event over OTLP.

- The SIEM/log store is immutable and append-only (MIR-06).
- Anomaly detection maintains per-agent baselines and swarm-campaign correlation (MIR-02).
- Every audit event carries: agent ID, session ID, **correlation ID**, timestamp, action, target, verdict, and the originating user session.

The payoff: the correlation ID ties every event to the originating user session. The full chain of agent-7's request, including the four gate verdicts, is reconstructable from the audit log. Incident response is not a hunt; it is a query.

- IR playbooks cover prompt-injection containment, model rollback, and tool revocation (MIR-04).
- Reporting triggers pre-populate SOCI 12-hour, APRA 72-hour, and NDB notifications (MIR-05).

What the audience should take away: the same events that gate production are the evidence an auditor or regulator asks for. "Audit becomes a test run" is literal here: the compliance console (Zone 5, GRM-05) continuously tests the controls against the live deployment.

Act 6: The Fail-Closed Beat (3 Minutes)

The most memorable minute of the talk. Put the room in a scenario:

“The OPA container just went down. Agent-7 proposes its next action. What happens?”

Answer, delivered slowly: **the action is blocked. Not degraded, not deferred: blocked.** If any dependency in the chain is unreachable (the policy engine, the tool gateway, or the audit destination), the action is denied (EE-06, TP-03). The framework never degrades to a weaker enforcement mode. And the failed evaluation is itself an audit event, alerting monitoring (MIR-02).

Why this beat matters: it converts the architecture from description into a set of engineering decisions. The room stops mapping boxes and starts asking “what would my system do in that moment?” That is the interaction story, felt.

Presenter note: this is the beat to deliver without slides, standing still. Silence after the question. Then the answer.

Act 7: The Architecture for You (5 Minutes)

Close by turning the architecture into a decision each person in the room can make. The deployment topology depends on fleet size:

Fleet size	Topology	Notes
Under ~10 agents	Embedded library	Controls run inside the agent framework; simplest path for pilots and single-agent PoCs
10-100 agents	Sidecar proxy	Per-agent sidecar (OPA + tool gateway + telemetry) in a container-orchestrated deployment

Fleet size	Topology	Notes
100+ or multi-agent	Central gateway + sidecars	Central policy/ gateway for external tools; sidecars retained for per-agent policy and audit

Then the two-diagram handout: the request flow with decision points, and the network boundary with the blocked control planes (EC2, IAM, KMS, CloudFormation, S3 control-plane, Azure Resource Manager, GCP Cloud Resource Manager: all excluded from every allow-list by EE-02 / TP-08; any attempt to reach them is logged, blocked, and treated as a potential indicator of compromise).

The closing question: *"What would you build first?"* Invite each person to self-select their starting point: the sandbox (Zone 2), the gates (Zone 3), the audit trail (Zone 5), or the topology that fits their fleet.

Handout guidance

The five diagrams that do not appear on slides travel as handouts:

1. `impl-reference-architecture.svg`: the five-zone deployable component view
2. `impl-policy-decision-points.svg`: the four gates
3. `impl-sidecar-topology.svg`: sidecar deployment in a container orchestrator
4. `impl-telemetry-pipeline.svg`: fleet to SIEM to reporting
5. `impl-network-boundaries.svg`: public/agent/trusted service zones
6. `impl-maturity-components.svg`: component groups by maturity level

This document itself is the seventh handout: it carries the full narrative, the incident anchors, and this presenter's guidance.

The discipline rules (from the hats session)

1. **One through-line.** Agent-7's request. No scene changes without a reason.

2. **Two diagrams on slides.** Request flow + network boundary. The rest are handouts.
3. **Gloss every control ID.** “TP-08, which blocks cloud control planes”, never a naked code.
4. **Anchor every gate to an incident.** The incidents are the reason the components exist.
5. **Fail-closed is the payoff.** Tell it as a beat, not a bullet.
6. **Close with the topology tree.** The architecture, made personal.
7. **Narrate from memory.** If you cannot tell the story without the deck, the deck is not ready.